Parallel Link Based Ranking Algorithms

Sandhiya Shunmugavel
Department of Computer Science
North Carolina State University
Raleigh, North Carolina, USA
sshunmu2@ncsu.edu

Sanjaey Shunmuga Sundaram
Department of Computer Science
North Carolina State University
Raleigh, North Carolina, USA
sshunmu@ncsu.edu

Abstract

In an era dominated by vast and rapidly expanding web content, the efficiency and relevance of information retrieval systems have become paramount. This project investigates the parallelization of link-based ranking algorithms—specifically the Hyperlink-Induced Topic Search (HITS) and PageRank algorithms—using MPI (Message Passing Interface) and OpenMP. Both algorithms utilize the structure of hyperlinks to assess the relevance and importance of web pages, complementing content-based search methods. Given the massive scale of modern web graphs, sequential implementations of these algorithms are computationally expensive and inefficient. Through parallel execution, we significantly reduce computation time and demonstrate the scalability and performance benefits of parallel computing in processing web-scale data.

Keywords

Parallel Algorithms, PageRank, HITS, MPI, OpenMP, CUDA, Graph Processing, Web Mining, Link Analysis, Sparse Matrix Computation, GPU Acceleration, Distributed Systems, Shared Memory, Scalability, Network Analysis, Power Iteration Method, COO Format, Performance Optimization, Big Data Analytics, High Performance Computing, Load Balancing, Communication Overhead, Convergence Analysis

1 Introduction

Efficient retrieval and ranking of relevant web information are vital in today's data-driven world, where web graphs comprise billions of nodes and hyperlinks. Traditional search engines initially relied heavily on link-based ranking algorithms such as PageRank and HITS, which evaluate page importance based on their hyperlink structure. PageRank determines a page's relevance based on the quantity and quality of links, while HITS distinguishes between authoritative pages and hub pages.

However, the scale of modern web data poses substantial challenges for sequential algorithm implementations, leading to slow convergence and high computational costs. This necessitates the use of parallel computing techniques, which can leverage the power of multi-core processors and distributed memory systems to handle large datasets efficiently. In this project, we parallelized the HITS and PageRank algorithms using MPI and OpenMP to enhance performance and scalability. The results validate that parallelization significantly accelerates computation while maintaining the accuracy of the ranking results, thus proving its effectiveness in large-scale web data processing.

2 HITS Algorithm

The Hyperlink-Induced Topic Search (HITS) algorithm, proposed by Jon Kleinberg, ranks web pages based on the hyperlink structure of the web. It distinguishes two types of web pages:

- **Authority pages**: Pages that contain valuable information and are pointed to by many hubs.
- **Hub pages**: Pages that link to many authorities.

A good hub links to many good authorities, and a good authority is pointed to by many good hubs.

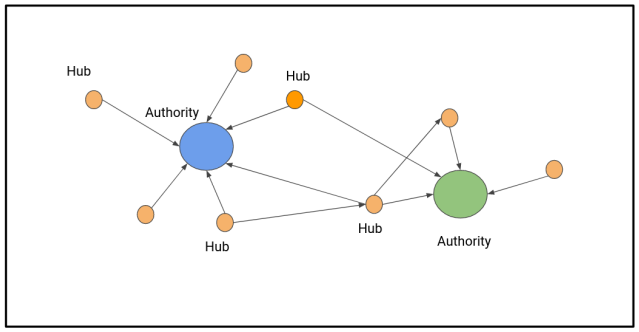


Figure 1: Hubs and Authority

Modeling the Web

The web is modeled as a directed graph where each node represents a web page and each edge represents a hyperlink from one page to another.

Initialization. Each node (page) is assigned an initial authority and hub score, typically set to 1.

Algorithm Iterations. Each iteration of the algorithm consists of two main steps:

(1) **Authority Update:** The authority score of a page is updated to be the sum of the hub scores of all pages that link to it.
(2) **Hub Update:** The hub score of a page is updated to be the sum of the authority scores of all pages it links to.

Mathematically, for each node i:

$$a_i = \sum_{j \in I(i)} h_j$$

$$h_i = \sum_{j \in O(i)} a_j$$

Where:

- a_i is the authority score of node i
- h_i is the hub score of node i
- $I(i)$ is the set of nodes linking to node i
- $O(i)$ is the set of nodes node i links to

Normalization. After each iteration, the authority and hub vectors are normalized (e.g., using Euclidean norm) to prevent scores from growing without bound and to ensure convergence.

Convergence. The process is repeated until the scores converge—typically, when the change in scores between iterations becomes smaller than a defined threshold or a maximum number of iterations is reached.

3 Page Rank Algorithm

3.1 Overview

PageRank is a link-based ranking algorithm originally developed by Larry Page and Sergey Brin as part of the foundation of the Google search engine. It revolutionized web search by introducing a way to rank web pages not just based on their content, but on their importance within the web's hyperlink structure.

The key idea behind PageRank is that a page is important if it is linked to by other important pages. In other words, incoming links are treated as votes of confidence, and not all votes are equal—being linked by a highly ranked page is more valuable than being linked by a low-ranked one.

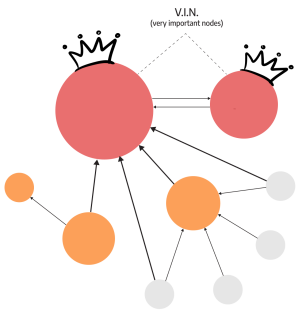


Figure 2: How Page rank - Decides which website is important

3.2 Working of Page Rank Algorithm

Modeling the Web. The web is represented as a directed graph where each page is a node and hyperlinks are directed edges.

Initialization. Each page is given an initial PageRank value, typically $\frac{1}{N}$, where N is the total number of pages.

Rank Distribution. In each iteration, a page distributes its PageRank equally to all pages it links to, based on its out-degree. The new PageRank of a page is the sum of the fractions received from all pages that link to it.

Damping Factor. A damping factor d (usually set to 0.85) is applied to simulate a user's probability of continuing to click on links. The PageRank formula is given by:

$$PR(i) = \frac{1-d}{N} + d \sum_{j \in B_i} \frac{PR(j)}{L(j)}$$

Where:

- $PR(i)$ is the PageRank of page i
- B_i is the set of pages linking to page i
- $L(j)$ is the number of outbound links from page j
- d is the damping factor
- N is the total number of pages

Convergence. The process is repeated until the PageRank values converge, meaning the changes between iterations fall below a predefined threshold.

4 Parallel HITS Implementation Using MPI

The function `run_hits()` implements the HITS algorithm in parallel using MPI for distributed systems. It operates on a graph stored in COO (Coordinate List) format and updates authority and hub scores for each node.

Initialization.

- N is set to the number of nodes in the graph.
- Arrays `new_auth` and `new_hub` are allocated and initialized to zero.
- All hub scores are initialized to 1.0.
- The node space is divided among processes: each MPI process handles a chunk defined by its `rank`.

Iteration Loop. For each iteration until `max_iter`:

(1) **Authority Update**:
- Each process sets `new_auth` values for its chunk to 0.
- It loops over all non-zeros in the COO matrix and accumulates contributions to the authority score from incoming hub values.
- `MPI_Allreduce` is used to combine local authority scores across all processes into `global_auth`.
- The authority vector is normalized using a helper function.

(2) **Hub Update**:
- Similarly, `new_hub` is initialized for the chunk.
- For each non-zero entry where the row index belongs to the process's chunk, the hub score is updated using `global_auth` values.
- Another `MPI_Allreduce` combines hub scores globally into `global_hub`.
- The hub vector is also normalized.

(3) **Convergence Check**:
- Each process computes the sum of absolute differences between new and old scores.
- A final `MPI_Allreduce` calculates the global difference.
- If the difference is below a predefined `EPSILON`, the algorithm converges and terminates early.

(4) **Score Update**:
- The new authority and hub scores are copied into the original `auth` and `hub` arrays for the next iteration.

Initialization. The function starts by determining the number of nodes and allocating memory:

```
int N = coo->num_rows;
float* new_auth = (float*)calloc(N, sizeof(float))
    ;
float* new_hub = (float*)calloc(N, sizeof(float));

for (int i = 0; i < N; ++i)
    hub[i] = 1.0;
```

- N is the number of nodes in the graph.
- new_auth and new_hub store intermediate values for each iteration.
- All hub scores are initialized to 1.

The dataset is split evenly across all MPI processes:

```
int chunk = N / size;
int start = rank * chunk;
int end = (rank == size - 1) ? N : start + chunk;
```

Each process works on its assigned chunk defined by its rank.

Iteration Loop. The algorithm iterates until a maximum number of iterations or convergence:

```
for (int iter = 0; iter < max_iter; ++iter) {
```

Step 1: Authority Update. Initialize local authority values:

```
for (int i = start; i < end; ++i)
    new_auth[i] = 0.0;
```

Update authority using incoming hub values:

```
for (int i = 0; i < coo->num_nonzeros; ++i)
    if (coo->cols[i] >= start && coo->cols[i] <
        end)
        new_auth[coo->cols[i]] += hub[coo->rows[i
            ]];
```

Combine values across all processes:

```
float* global_auth = (float*)calloc(N, sizeof(
    float));
MPI_Allreduce(new_auth, global_auth, N, MPI_FLOAT,
     MPI_SUM, comm);
normalize_vector(global_auth, N, comm);
```

Step 2: Hub Update. Reset local hub scores:

```
for (int i = start; i < end; ++i)
    new_hub[i] = 0.0;
```

Update using outgoing authority scores:

```
for (int i = 0; i < coo->num_nonzeros; ++i)
    if (coo->rows[i] >= start && coo->rows[i] <
        end)
        new_hub[coo->rows[i]] += global_auth[coo->
            cols[i]];
```

Combine hub values globally:

```
float* global_hub = (float*)calloc(N, sizeof(float
    ));
MPI_Allreduce(new_hub, global_hub, N, MPI_FLOAT,
    MPI_SUM, comm);
normalize_vector(global_hub, N, comm);
```

Step 3: Convergence Check. Calculate the total difference between the new and previous iteration:

```
float local_diff = 0.0, global_diff = 0.0;
for (int i = 0; i < N; ++i)
    local_diff += fabs(global_auth[i] - auth[i]) +
         fabs(global_hub[i] - hub[i]);
MPI_Allreduce(&local_diff, &global_diff, 1,
    MPI_FLOAT, MPI_SUM, comm);
```

Print convergence info:

```
if (rank == 0)
    printf("HITS Iteration %d, diff = %.6f\n",
        iter + 1, global_diff);
```

If convergence criterion is met, exit early:

```
if (global_diff < EPSILON) {
    free(global_auth);
    free(global_hub);
    break;
}
```

Step 4: Score Update. Copy updated values into auth and hub arrays:

```
for (int i = 0; i < N; ++i) {
    auth[i] = global_auth[i];
    hub[i] = global_hub[i];
}
```

Free intermediate buffers:

```
free(global_auth);
free(global_hub);
```

Final Cleanup. After exiting the loop, the final temporary arrays are deallocated:

```
free(new_auth);
free(new_hub);
}
```

5 Parallel PageRank Implementation Using MPI

This section describes the PageRank algorithm implemented using MPI. The graph is stored in COO (Coordinate List) format. Each process updates a subset of the nodes and uses collective communication to compute the global PageRank vector.

Helper Function: Vector Normalization. This function normalizes a vector using its Euclidean (L2) norm across all MPI processes.

```
void normalize_vector(float* vec, int size,
    MPI_Comm comm) {
    float local_sum = 0.0, global_sum = 0.0;
    for (int i = 0; i < size; ++i)
        local_sum += vec[i] * vec[i];
    MPI_Allreduce(&local_sum, &global_sum, 1,
        MPI_FLOAT, MPI_SUM, comm);
    global_sum = sqrt(global_sum);
    if (global_sum == 0) return;
    for (int i = 0; i < size; ++i)
        vec[i] /= global_sum;
}
```

Listing 1: Vector Normalization

Initialization. We start by initializing important data structures and computing out-degrees:

```
int N = coo->num_rows;
float* new_pr = (float*)calloc(N, sizeof(float));
int* out_degree = (int*)calloc(N, sizeof(int));

for (int i = 0; i < coo->num_nonzeros; ++i)
    out_degree[coo->rows[i]]++;

for (int i = 0; i < N; ++i)
    pr[i] = 1.0 / N;
```

- N is the total number of nodes.
- `out_degree` holds the count of outgoing links for each node.
- Each node starts with an initial PageRank value of $1/N$.

Process Partitioning. The node space is divided among MPI processes:

```
int chunk = N / size;
int start = rank * chunk;
int end = (rank == size - 1) ? N : start + chunk;
```

Each process works on nodes in the range `[start, end)`.

PageRank Iteration Loop. The algorithm iteratively updates the PageRank values:

```
for (int iter = 0; iter < max_iter; ++iter) {
```

Step 1: Base Rank Initialization. Each process initializes its portion of the new PageRank vector:

```
for (int i = start; i < end; ++i)
    new_pr[i] = (1.0 - damping) / N;
```

This term handles the random teleportation factor.

Step 2: Rank Distribution. Each process computes rank contributions from incoming links:

```
for (int i = 0; i < coo->num_nonzeros; ++i) {
    int src = coo->rows[i];
    int dst = coo->cols[i];
    if (dst >= start && dst < end && out_degree[
        src] > 0)
        new_pr[dst] += damping * pr[src] /
            out_degree[src];
}
```

Step 3: Global Rank Aggregation. The partial vectors are combined across processes:

```
float* global_pr = (float*)calloc(N, sizeof(float)
    );
MPI_Allreduce(new_pr, global_pr, N, MPI_FLOAT,
    MPI_SUM, comm);
```

Step 4: Convergence Check. The difference from the previous iteration is computed:

```
float local_diff = 0.0, global_diff = 0.0;
for (int i = 0; i < N; ++i)
    local_diff += fabs(global_pr[i] - pr[i]);
MPI_Allreduce(&local_diff, &global_diff, 1,
    MPI_FLOAT, MPI_SUM, comm);
```

The master process prints the difference:

```
if (rank == 0)
    printf("PageRank Iteration %d, diff = %.6f\n",
        iter + 1, global_diff);
```

If converged, the loop exits early:

```
if (global_diff < EPSILON) {
    free(global_pr);
    break;
}
```

Step 5: Update Scores. The new values are copied for the next iteration:

```
for (int i = 0; i < N; ++i)
    pr[i] = global_pr[i];
free(global_pr);
```

Final Cleanup. Once all iterations are done, the dynamic memory is freed:

```
free(new_pr);
free(out_degree);
}
```

6 Parallel HITS Implementation Using OpenMP

In each iteration, the authority scores are updated by multiplying the transpose of the adjacency matrix A^T with the hub vector, and the hub scores are updated by multiplying A with the new authority vector.

To efficiently parallelize this on shared memory systems, we use OpenMP:

- Arrays are initialized in parallel using `#pragma omp parallel for`.
- Matrix-vector multiplications are parallelized over non-zero entries in the COO format.

- Since multiple threads may write to the same output entry (e.g., `new_auth[col]`), we use `#pragma omp atomic` to ensure correctness.
- The convergence criterion is checked using a reduction clause to accumulate the total difference across threads.

The function `run_hits()` implements the HITS algorithm in parallel using OpenMP for shared-memory systems. The graph is stored in COO (Coordinate List) format, and the algorithm iteratively updates hub and authority scores for each node.

Initialization.

- N is the number of nodes in the graph.
- Arrays `auth`, `hub`, `new_auth`, and `new_hub` are dynamically allocated and initialized.
- Hub scores are initialized to 1.0 using a parallel loop.

```
int N = coo->num_rows;

float* auth = (float*)calloc(N, sizeof(float));
float* hub = (float*)calloc(N, sizeof(float));
float* new_auth = (float*)calloc(N, sizeof(float))
    ;
float* new_hub = (float*)calloc(N, sizeof(float));

#pragma omp parallel for
for (int i = 0; i < N; ++i)
    hub[i] = 1.0;
```

Iteration Loop. The algorithm proceeds for a maximum of `max_iter` iterations or until convergence. In each iteration:

Step 1: Reset Intermediate Arrays. All entries of `new_auth` and `new_hub` are reset to 0.0 in parallel:

```
#pragma omp parallel for
for (int i = 0; i < N; ++i) {
    new_auth[i] = 0.0;
    new_hub[i] = 0.0;
}
```

Step 2: Authority Update ($a = A^T \cdot h$). Each non-zero entry in the COO matrix contributes to the authority score of its column. This is done in parallel with atomic updates:

```
#pragma omp parallel for
for (int i = 0; i < coo->num_nonzeros; ++i) {
    #pragma omp atomic
    new_auth[coo->cols[i]] += hub[coo->rows[i]];
}
normalize_vector(new_auth, N);
```

Step 3: Hub Update ($h = A \cdot a$). Each non-zero entry contributes to the hub score of its row:

```
#pragma omp parallel for
for (int i = 0; i < coo->num_nonzeros; ++i) {
    #pragma omp atomic
    new_hub[coo->rows[i]] += new_auth[coo->cols[i
        ]];
}
normalize_vector(new_hub, N);
```

Step 4: Convergence Check. The total difference between the new and previous scores is computed using a parallel reduction:

```
float diff = 0.0;
#pragma omp parallel for reduction(+:diff)
for (int i = 0; i < N; ++i)
    diff += fabs(new_auth[i] - auth[i]) + fabs(
        new_hub[i] - hub[i]);

fprintf(out, "Iteration %d, diff = %.6f\n", iter +
     1, diff);
if (diff < EPSILON)
    break;
```

Step 5: Swap Buffers. Pointers to the new scores are swapped with the original arrays for the next iteration:

```
float* temp = auth; auth = new_auth; new_auth =
    temp;
temp = hub; hub = new_hub; new_hub = temp;
```

Final Output and Cleanup. After the loop finishes, the final scores are printed and memory is deallocated:

```
fprintf(out, "\nFinal Authority Scores (full
    precision):\n");
for (int i = 0; i < N; ++i)
    fprintf(out, "Node %d -> Auth: %.10f\n", i,
        auth[i]);

fprintf(out, "\nFinal Hub Scores (full precision)
    :\n");
for (int i = 0; i < N; ++i)
    fprintf(out, "Node %d -> Hub: %.10f\n", i, hub
        [i]);

free(auth); free(hub); free(new_auth); free(
    new_hub);
```

7 Parallel PageRank Implementation Using OpenMP

The function `run_pagerank()` implements the PageRank algorithm in parallel using OpenMP. The graph is stored in COO (Coordinate List) format. The algorithm updates the PageRank score for each node iteratively using the power method.

Initialization.

- N is the total number of nodes in the graph.
- The `pr` and `new_pr` arrays store current and next PageRank scores, respectively.
- The `out_degree` array is initialized to count outgoing edges for each node.
- All PageRank scores are initialized uniformly to $1/N$.

```
int N = coo->num_rows;
float* pr = (float*)malloc(N * sizeof(float));
float* new_pr = (float*)malloc(N * sizeof(float));
int* out_degree = (int*)calloc(N, sizeof(int));

#pragma omp parallel for
for (int i = 0; i < coo->num_nonzeros; ++i)
```

```
    #pragma omp atomic
    out_degree[coo->rows[i]]++;

#pragma omp parallel for
for (int i = 0; i < N; ++i)
    pr[i] = 1.0 / N;
```

Iteration Loop. The algorithm runs for up to `max_iter` iterations or until the PageRank vector converges.

Step 1: Reset New PageRank Vector. Each node is assigned a base rank that accounts for teleportation (random jump):

```
#pragma omp parallel for
for (int i = 0; i < N; ++i)
    new_pr[i] = (1.0 - damping) / N;
```

Step 2: Distribute Rank Contributions. Each node distributes its PageRank score to its outgoing neighbors proportionally. This step is parallelized, and atomic operations are used to avoid race conditions on `new_pr`:

```
#pragma omp parallel for
for (int i = 0; i < coo->num_nonzeros; ++i) {
    int src = coo->rows[i];
    int dst = coo->cols[i];
    if (out_degree[src] > 0) {
        float contrib = damping * pr[src] /
            out_degree[src];
        #pragma omp atomic
        new_pr[dst] += contrib;
    }
}
```

Step 3: Convergence Check. The algorithm checks the L1-norm difference between the old and new PageRank vectors:

```
float diff = 0.0;
#pragma omp parallel for reduction(+:diff)
for (int i = 0; i < N; ++i)
    diff += fabs(new_pr[i] - pr[i]);

fprintf(out, "PageRank Iteration %d, diff = %.6f\n
    ", iter + 1, diff);
if (diff < EPSILON)
    break;
```

Step 4: Swap Buffers. The new values are swapped with the current PageRank vector:

```
float* temp = pr;
pr = new_pr;
new_pr = temp;
```

Final Output and Cleanup. At the end of the loop, final PageRank values are printed, and all allocated memory is freed:

```
fprintf(out, "\nFinal PageRank Scores:\n");
for (int i = 0; i < N; ++i)
    fprintf(out, "Node %d -> PR: %.10f\n", i, pr[i
        ]);

free(pr); free(new_pr); free(out_degree);
```

8 Parallel HITS Implementation Using CUDA

The HITS algorithm is parallelized for GPU execution using CUDA. Each update step of the hub and authority scores is offloaded to GPU kernels. The sparse graph is stored in Coordinate (COO) format and operations are performed over non-zero entries.

Key CUDA Kernels.

- `hits_auth_kernel`: Computes authority scores by performing atomic additions from hubs to their linked authorities.
- `hits_hub_kernel`: Computes hub scores by aggregating authority values of connected nodes.
- `normalize_kernel`: Normalizes a given vector by dividing each entry by the L2 norm.
- `compute_diff_kernel`: Computes element-wise difference between vectors to check for convergence.
- `reset_vector`: Resets a given vector to a specified value.

Memory Allocation and Initialization.

- Host arrays `auth` and `hub` are allocated and initialized (`hub` to 1.0).
- Device memory is allocated for COO row and column arrays, vectors, and convergence buffer.
- Host COO arrays are copied to device memory.

```
cudaMalloc(&d_rows, NNZ * sizeof(int));
cudaMalloc(&d_cols, NNZ * sizeof(int));
cudaMalloc(&d_auth, N * sizeof(float));
cudaMalloc(&d_hub, N * sizeof(float));
cudaMalloc(&d_diff, sizeof(float));

cudaMemcpy(d_rows, coo->rows, NNZ * sizeof(int),
    cudaMemcpyHostToDevice);
cudaMemcpy(d_cols, coo->cols, NNZ * sizeof(int),
    cudaMemcpyHostToDevice);
```

Main Iteration Loop. The HITS algorithm runs for a maximum of `MAX_ITERATIONS` or until convergence:

Step 1: Authority Update ($a = A^T \cdot h$).

- `d_auth` is reset to zero.
- Each edge (row, col) contributes `hub[row]` to `auth[col]` using atomic addition.
- The result is normalized using the L2 norm.

```
reset_vector<<<numBlocks_N, blockSize>>>(d_auth,
    0.0f, N);
hits_auth_kernel<<<numBlocks_NNZ, blockSize>>>(
    d_rows, d_cols, d_hub, d_auth, NNZ);
normalize_kernel<<<numBlocks_N, blockSize>>>(
    d_auth, N, norm_auth);
```

Step 2: Hub Update ($h = A \cdot a$).

- `d_hub` is reset to zero.
- Each edge (row, col) contributes `auth[col]` to `hub[row]` using atomic addition.
- The result is normalized similarly.

```
reset_vector<<<numBlocks_N, blockSize>>>(d_hub,
    0.0f, N);
hits_hub_kernel<<<numBlocks_NNZ, blockSize>>>(
    d_rows, d_cols, d_auth, d_hub, NNZ);
normalize_kernel<<<numBlocks_N, blockSize>>>(d_hub
    , N, norm_hub);
```

Step 3: Convergence Check. The algorithm checks if the L1 norm of difference between new and previous values of auth and hub vectors is below a threshold:

```
cudaMemcpy(d_diff, &host_diff, sizeof(float),
    cudaMemcpyHostToDevice);
compute_diff_kernel<<<numBlocks_N, blockSize>>>(
    d_auth, auth, d_diff, N);
compute_diff_kernel<<<numBlocks_N, blockSize>>>(
    d_hub, hub, d_diff, N);
cudaMemcpy(&host_diff, d_diff, sizeof(float),
    cudaMemcpyDeviceToHost);
if (host_diff < EPSILON) break;
```

Step 4: Host Updates. The new device vectors are copied back to host memory for the next iteration:

```
cudaMemcpy(auth, d_auth, N * sizeof(float),
    cudaMemcpyDeviceToHost);
cudaMemcpy(hub, d_hub, N * sizeof(float),
    cudaMemcpyDeviceToHost);
```

Final Output and Cleanup. At the end of execution, execution time is measured and scores are written to file. All GPU and CPU memory is released.

```
fprintf(output_file, "HITS: %.4f", duration);
free(auth); free(hub);
cudaFree(d_rows); cudaFree(d_cols);
cudaFree(d_auth); cudaFree(d_hub); cudaFree(d_diff
    );
```

9 Parallel PageRank Implementation Using CUDA

The PageRank algorithm is implemented using CUDA for GPU parallelization. The graph is stored in Coordinate List (COO) format and PageRank scores are iteratively updated based on the power iteration method.

Key CUDA Kernels.

- pagerank_kernel: Distributes PageRank score from each node to its neighbors using atomic updates.
- reset_vector: Initializes the new_pr array with teleportation value.
- compute_diff_kernel: Calculates the element-wise difference between current and previous vectors to check for convergence.

Memory Allocation and Initialization.

- Host arrays pr and new_pr store PageRank scores.
- out_degree is computed on host to count outgoing edges per node.
- GPU memory is allocated and initialized with COO structure, PageRank vectors, and convergence buffer.

```
pr = (float*)malloc(N * sizeof(float));
out_degree = (int*)calloc(N, sizeof(int));
for (int i = 0; i < NNZ; ++i)
    out_degree[coo->rows[i]]++;
for (int i = 0; i < N; ++i)
    pr[i] = 1.0f / N;

cudaMalloc(&d_out_degree, N * sizeof(int));
cudaMemcpy(d_out_degree, out_degree, N * sizeof(
    int), cudaMemcpyHostToDevice);
```

Main Iteration Loop. The PageRank algorithm runs for MAX_ITERATIONS or until convergence.

Step 1: Reset Vector with Teleportation. Each entry of new_pr is initialized to $(1 - \text{DAMPING})/N$ to account for teleportation:

```
reset_vector<<<numBlocks_N, blockSize>>>(d_new_pr,
     (1.0f - DAMPING) / N, N);
```

Step 2: Rank Distribution. Each edge contributes to the PageRank score of its destination node using the current rank of the source node:

```
pagerank_kernel<<<numBlocks_NNZ, blockSize>>>(
    d_rows, d_cols, d_pr, d_new_pr, d_out_degree,
        NNZ, DAMPING, N);
```

Step 3: Convergence Check. The L1 norm difference between pr and new_pr is computed. If the difference is below a threshold, the loop terminates:

```
compute_diff_kernel<<<numBlocks_N, blockSize>>>(
    d_new_pr, d_pr, d_diff, N);
cudaMemcpy(&host_diff, d_diff, sizeof(float),
    cudaMemcpyDeviceToHost);
if (host_diff < EPSILON) break;
```

Step 4: Swap Buffers. The updated PageRank vector new_pr is copied back to host memory for the next iteration:

```
cudaMemcpy(pr, d_new_pr, N * sizeof(float),
    cudaMemcpyDeviceToHost);
```

Final Output and Cleanup. The total execution time is recorded using CUDA events and written to output. All host and device memory is released.

```
cudaEventElapsedTime(&milliseconds, startEvent,
    stopEvent);
fprintf(output_file, "PageRank: %.4f",
    milliseconds / 1000.0);

free(pr); free(new_pr); free(out_degree);
cudaFree(d_rows); cudaFree(d_cols); cudaFree(d_pr)
    ;
cudaFree(d_new_pr); cudaFree(d_out_degree);
    cudaFree(d_diff);
cudaEventDestroy(startEvent); cudaEventDestroy(
    stopEvent);
```

10 Dataset Description and Role

The dataset used in this project represents a **web link structure**, typically encoded as a directed graph, where:

Nodes represent individual web pages. **Edges** (directed links) represent hyperlinks from one page to another.

- **web-Google**:
  - Contains 875,713 nodes and 5,105,039 edges.
  - A directed web graph built from crawled hyperlinks on the Google domain.
  - Ideal for simulating the PageRank "random surfer" model and for analyzing hub-authority structure using HITS.
- **wiki-Talk**:
  - Contains 2,394,385 nodes and 5,021,410 edges.
  - A directed communication network based on user-to-user interactions in Wikipedia talk pages.
  - Captures influence flow dynamics and is well-suited for authority/hub analysis with HITS.
- **Orkut Network**:
  - Contains 3,072,441 nodes and 117,185,083 edges.
  - An undirected social graph extracted from the Orkut social network.
  - Features a dense community structure, making it excellent for testing algorithm scalability in massive social networks.
- **ca-GrQc**:
  - Contains 5,242 nodes and 14,496 edges.
  - An undirected collaboration network of authors from the General Relativity and Quantum Cosmology category on arXiv.
  - Exhibits a well-defined hub-authority structure, making it perfect for HITS demonstrations on smaller graphs.

This graph-based data set is used to evaluate link-based ranking algorithms such as HITS and PageRank.

11 Performance Comparison of HITS: MPI vs OpenMP vs CUDA

The figure below shows the execution time (in seconds, log scale) of the HITS algorithm across different datasets using three parallelization strategies. MPI, OpenMP, and CUDA.

- **CUDA achieves the best performance in all datasets**, consistently outperforming MPI and OpenMP. This is due to the massively parallel architecture and memory throughput of GPUs.
- On the largest dataset, **Orkut**, the performance gap is substantial:
  - **MPI:** 300.822 seconds
  - **OpenMP:** 98.473 seconds
  - **CUDA: 0.177 seconds**

  CUDA provides over 500x speedup compared to MPI.
- On **Web-Google** and **Wiki-Talk**:

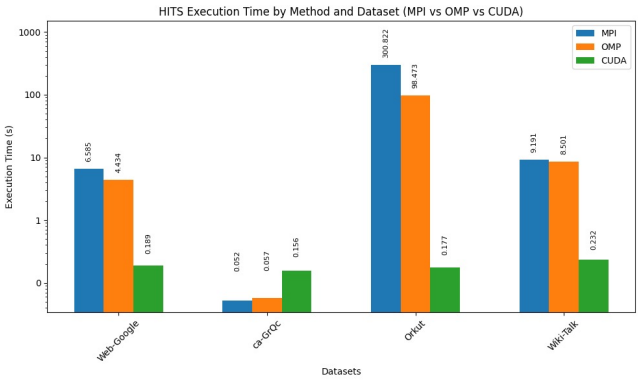


Figure 3: HITS Execution Time by Method and Dataset (MPI vs OMP vs CUDA)

  - Web-Google: CUDA (0.189 s) vs OpenMP (4.434 s) and MPI (6.585 s)
  - Wiki-Talk: CUDA (0.232 s) vs OpenMP (8.501 s) and MPI (9.191 s)

  CUDA demonstrates over 30x speedup on these mid-sized graphs.
- On the smaller graph **ca-GrQC**, differences are not that noticable, but CUDA still outperforms:
  - CUDA: 0.156 s, OpenMP: 0.057 s, MPI: 0.052 s

  This indicates that CUDA's advantage becomes more significant as the graph size increases.
- The **logarithmic y-axis** scale emphasizes the exponential run-time reduction achieved by CUDA.

CUDA provides the most efficient execution for the HITS algorithm, with performance benefits scaling with dataset size. While OpenMP improves over MPI due to lower communication overhead, GPU acceleration delivers orders-of-magnitude speedups, especially on large sparse graphs.

12 Performance Comparison of PageRank: MPI vs OpenMP vs CUDA

The figure below illustrates the execution time (in seconds, logarithmic scale) of the PageRank algorithm across four datasets using MPI, OpenMP, and CUDA-based parallelization strategies.

- **CUDA provides the fastest execution time** in all data sets, highlighting the effectiveness of GPU acceleration for iterative graph computations.
- On the large-scale **Orkut** dataset:
  - MPI: 109.152 s, OpenMP: 51.907 s, CUDA: **0.093 s**

  CUDA achieves more than a **1000x speed-up** over MPI.
- For **Web-Google**:
  - CUDA (0.089 s) is significantly faster than OpenMP (2.066 s) and MPI (4.445 s), giving over **40x speedup**.
- In **Wiki-Talk**:
  - MPI: 2.262 s, OpenMP: 1.514 s, CUDA: **0.004 s**

  CUDA provides an astonishing **more than 500x improvement**.

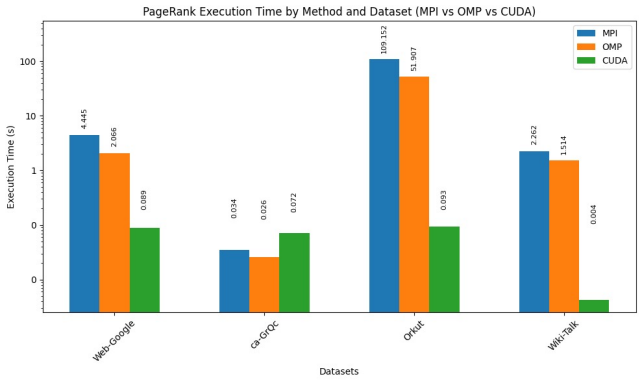


Figure 4: PageRank Execution Time by Method and Dataset (MPI vs OMP vs CUDA)

- For the smaller **ca-GrQC** graph, performance gains are less extreme:
  - MPI: 0.034 s, OpenMP: 0.026 s, CUDA: 0.072 s

  CUDA is slightly slower here due to kernel launch and memory transfer overheads, showing that for small graphs, CPU methods may remain competitive.
- The **log-scale y-axis** further emphasizes the exponential gains from CUDA for larger datasets.

CUDA demonstrates substantial acceleration for PageRank, particularly on large and sparse graphs. While OpenMP improves upon MPI by reducing inter-process communication overhead, GPU-based computation far exceeds CPU-based methods in both speed and scalability. For real-time or large-scale graph analytics, CUDA is the preferred choice.

13 Final Results and Conclusion

In this study, we implemented and evaluated parallel versions of two prominent link analysis algorithms—HITS and PageRank—using three parallel computing models: MPI (Message Passing Interface), OpenMP, and CUDA. The aim was to assess the scalability and performance of each method across various real-world graph datasets of different sizes and complexities. All graph inputs were stored in COO (Coordinate List) format to maintain consistency across implementations.

Execution Time Summary

Table 1 provides the execution times (in seconds) of both HITS and PageRank algorithms across four datasets, comparing Serial, MPI, OpenMP, and CUDA implementations.

Discussion of Results. The execution times in Table 1 align closely with the trends observed in the visual plots. Several key patterns emerge from the data:

- **CUDA consistently delivers the fastest performance**, achieving significant speedups over CPU-based approaches, particularly for large datasets like Orkut and Wiki-Talk. For example, HITS on Orkut takes 352.13 seconds in Serial but only 0.177 seconds with CUDA—a more than **1989x speedup**.

Table 1: Execution Time (in seconds) for HITS and PageRank across Datasets and Methods

Dataset	Algorithm	Serial	MPI	OpenMP	CUDA
Web-Google	HITS	9.653	6.585	4.434	**0.189**
	PageRank	3.259	4.445	2.066	**0.089**
ca-GrQc	HITS	0.057	0.052	0.057	**0.156**
	PageRank	0.029	0.034	0.026	**0.072**
Orkut	HITS	352.133	300.822	98.473	**0.177**
	PageRank	93.120	109.152	51.907	**0.093**
Wiki-Talk	HITS	8.730	9.191	8.501	**0.232**
	PageRank	2.355	2.262	1.514	**0.004**

- **OpenMP outperforms MPI and Serial** on all datasets except the smallest (ca-GrQc), where the gains are minimal due to insufficient workload to amortize parallel overheads.
- **Serial implementations remain competitive only on very small datasets** (e.g., ca-GrQc), where the cost of thread management or kernel launches in parallel methods may outweigh the benefits.
- On the **Web-Google** dataset, MPI-based PageRank (4.445 s) is actually slower than the Serial version (3.259 s), illustrating how communication overhead in MPI can negate the advantages of distributed parallelism for certain graphs.
- **GPU acceleration benefits scale significantly with graph size**. CUDA's architecture, designed for throughput over latency, handles large sparse matrix operations with high efficiency, making it ideal for iterative algorithms like HITS and PageRank.

Final Insights. Our evaluation shows that:

- **CUDA is the most scalable and efficient method** for large-scale graph analysis when GPU resources are available.
- **OpenMP offers an accessible and effective CPU-side solution**, especially on mid-sized graphs in shared-memory environments.
- **MPI is best suited for distributed memory systems**, but suffers from communication bottlenecks in smaller-scale or tightly-coupled workloads.

Choosing the appropriate parallelization strategy depends on several factors:

- Size and sparsity of the dataset
- Availability of hardware resources (GPU vs CPU clusters)
- Trade-offs between communication overhead and computational intensity
- Programming complexity and maintainability of code

This comparative analysis highlights that **hardware-aware design and algorithmic parallelism** are critical to unlocking the full potential of modern architectures for large-scale graph processing tasks.